

Project: Deep Learning Image Classification

This project focuses on image classification using Deep Learning. The CIFAR-10 dataset is used to classify images into 10 different categories. A pretrained ResNet18 model is used with transfer learning to improve the classification process. Image augmentation and preprocessing are applied before training the model.

✓ Objectives

1. To build an image classification model using Deep Learning.
2. To use the CIFAR-10 dataset for image classification.
3. To apply image preprocessing and data augmentation techniques.
4. To use transfer learning with a pretrained ResNet18 model.
5. To train and evaluate the model using appropriate performance metrics.
6. To visualize training accuracy and training loss.
7. To test the model on new images and save the trained model for future use.

```
import torch
import torchvision
import torchvision.transforms as transforms
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import numpy as np

print("Libraries imported successfully!")
print("PyTorch version:", torch.__version__)
print("Torchvision version:", torchvision.__version__)
```

```
Libraries imported successfully!
PyTorch version: 2.11.0+cu128
Torchvision version: 0.26.0+cu128
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("GPU is not available. Training will use CPU.")
```

```
Device: cuda
GPU: Tesla T4
```

```

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

print("Image transformations created successfully!")

```

Image transformations created successfully!

```

train_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=train_transform
)

test_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=test_transform
)

print("Training images:", len(train_dataset))
print("Testing images:", len(test_dataset))

```

100%|██████████| 170M/170M [31:30<00:00, 90.2kB/s]
 Training images: 50000
 Testing images: 10000

```

classes = [
    "airplane",
    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck"
]

```

```
print("Classes:")
print(classes)
```

Classes:
['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',

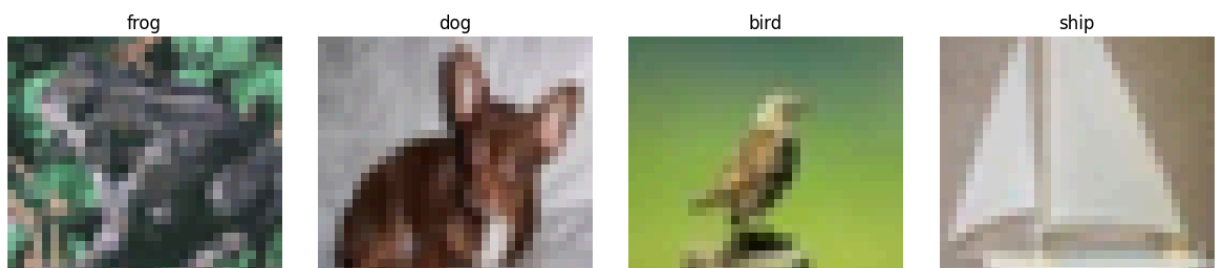
```
display_dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=False,
    transform=transforms.ToTensor()
)

images, labels = next(iter(
    torch.utils.data.DataLoader(
        display_dataset,
        batch_size=8,
        shuffle=True
    )
))

plt.figure(figsize=(12, 6))

for i in range(8):
    plt.subplot(2, 4, i + 1)
    plt.imshow(images[i].permute(1, 2, 0))
    plt.title(classes[labels[i]])
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)
```

```
print("DataLoaders created successfully!")
```

DataLoaders created successfully!

```
from torchvision.models import resnet18, ResNet18_Weights
weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
print("Pretrained ResNet18 loaded successfully!")
```

Pretrained ResNet18 loaded successfully!

```
for param in model.parameters():
    param.requires_grad = False

print("Pretrained layers frozen.")
```

Pretrained layers frozen.

```
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 10)
model = model.to(device)

print("Final classification layer replaced.")
print(model.fc)
```

Final classification layer replaced.
Linear(in_features=512, out_features=10, bias=True)

```
criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.fc.parameters(),
    lr=0.001
)

print("Loss function and optimizer created.")
```

Loss function and optimizer created.

```
num_epochs = 5

train_losses = []
train_accuracies = []

for epoch in range(num_epochs):

    model.train()

    running_loss = 0.0
```

```

correct = 0
total = 0

for images, labels in train_loader:

    images = images.to(device)
    labels = labels.to(device)

    optimizer.zero_grad()

    outputs = model(images)

    loss = criterion(outputs, labels)

    loss.backward()
    optimizer.step()

    running_loss += loss.item()

    _, predicted = torch.max(outputs, 1)

    total += labels.size(0)
    correct += (predicted == labels).sum().item()

epoch_loss = running_loss / len(train_loader)
epoch_accuracy = 100 * correct / total

train_losses.append(epoch_loss)
train_accuracies.append(epoch_accuracy)

print(
    f"Epoch [{epoch+1}/{num_epochs}] "
    f"Loss: {epoch_loss:.4f} "
    f"Accuracy: {epoch_accuracy:.2f}%"
)

```

```

Epoch [1/5] Loss: 0.6583 Accuracy: 77.18%
Epoch [2/5] Loss: 0.6559 Accuracy: 77.26%
Epoch [3/5] Loss: 0.6533 Accuracy: 77.54%
Epoch [4/5] Loss: 0.6508 Accuracy: 77.56%
Epoch [5/5] Loss: 0.6453 Accuracy: 77.84%

```

```

model.eval()

correct = 0
total = 0

all_predictions = []
all_labels = []

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

```

```

_, predicted = torch.max(outputs, 1)

total += labels.size(0)
correct += (predicted == labels).sum().item()

all_predictions.extend(predicted.cpu().numpy())
all_labels.extend(labels.cpu().numpy())

test_accuracy = 100 * correct / total

print(f"Test Accuracy: {test_accuracy:.2f}%")

```

Test Accuracy: 80.57%

```

from sklearn.metrics import classification_report, confusion_matrix

print(
    classification_report(
        all_labels,
        all_predictions,
        target_names=classes
    )
)

```

	precision	recall	f1-score	support
airplane	0.78	0.85	0.81	1000
automobile	0.87	0.88	0.88	1000
bird	0.79	0.71	0.75	1000
cat	0.79	0.62	0.70	1000
deer	0.69	0.81	0.75	1000
dog	0.77	0.78	0.77	1000
frog	0.78	0.88	0.83	1000
horse	0.87	0.76	0.81	1000
ship	0.86	0.87	0.87	1000
truck	0.87	0.88	0.87	1000
accuracy			0.81	10000
macro avg	0.81	0.81	0.80	10000
weighted avg	0.81	0.81	0.80	10000

```

cm = confusion_matrix(all_labels, all_predictions)

plt.figure(figsize=(10, 8))

plt.imshow(cm)

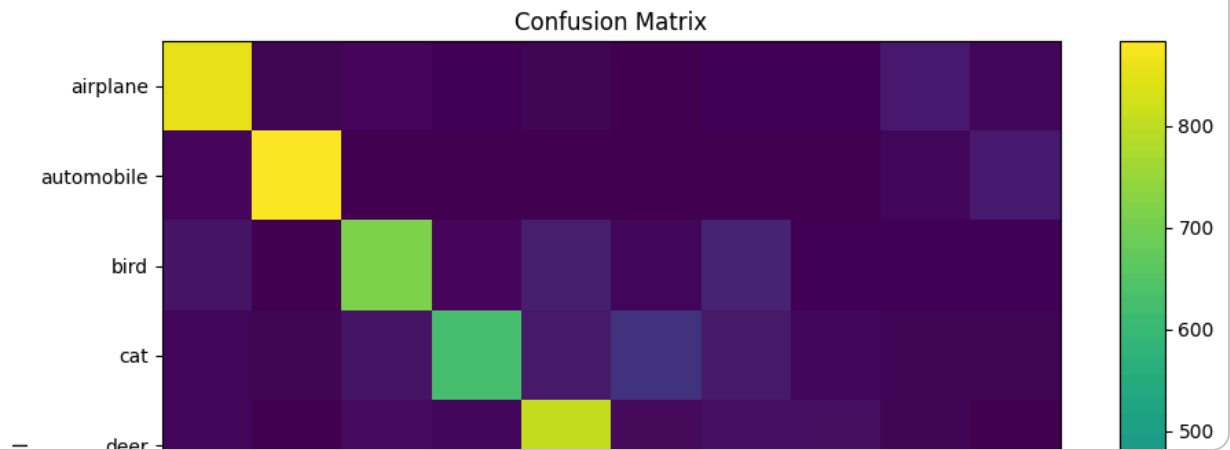
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.xticks(range(10), classes, rotation=45)
plt.yticks(range(10), classes)

plt.colorbar()

```

```
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, num_epochs + 1),
    train_accuracies,
    marker="o"
)

plt.xlabel("Epoch")
plt.ylabel("Training Accuracy (%)")
plt.title("Training Accuracy")

plt.grid(True)
plt.show()
```



```
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, num_epochs + 1),
    train_losses,
    marker="o"
)

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss")

plt.grid(True)
plt.show()
```



```
model.eval()

image, true_label = test_dataset[0]

with torch.no_grad():
    output = model(image.unsqueeze(0).to(device))
    predicted_label = torch.argmax(output, dim=1).item()

plt.figure(figsize=(5, 5))

display_image = image.cpu()

display_image = display_image * torch.tensor(
    [0.229, 0.224, 0.225]
).view(3, 1, 1)

display_image = display_image + torch.tensor(
    [0.485, 0.456, 0.406]
).view(3, 1, 1)

display_image = torch.clamp(display_image, 0, 1)

plt.imshow(display_image.permute(1, 2, 0))

plt.title(
    f"True: {classes[true_label]} | "
    f"Predicted: {classes[predicted_label]}"
)

plt.axis("off")
plt.show()
```

True: cat | Predicted: cat



```
model.eval()

images, labels = next(iter(test_loader))

images = images.to(device)
labels = labels.to(device)

with torch.no_grad():
    outputs = model(images)
    predictions = torch.argmax(outputs, dim=1)

plt.figure(figsize=(12, 8))

for i in range(8):

    image = images[i].cpu()

    image = image * torch.tensor(
        [0.229, 0.224, 0.225]
    ).view(3, 1, 1)

    image = image + torch.tensor(
        [0.485, 0.456, 0.406]
    ).view(3, 1, 1)

    image = torch.clamp(image, 0, 1)

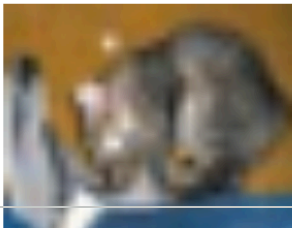
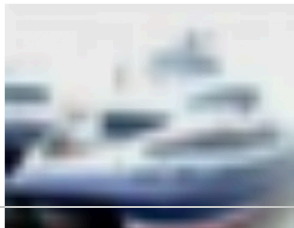
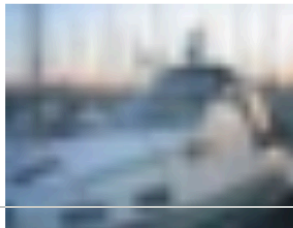
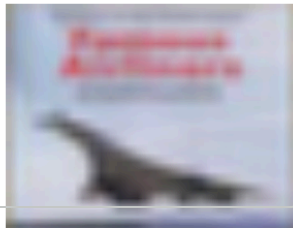
    plt.subplot(2, 4, i + 1)

    plt.imshow(image.permute(1, 2, 0))

    plt.title(
        f"True: {classes[labels[i].item()]}\\n"
        f"Pred: {classes[predictions[i].item()]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```

True: cat Pred: cat	True: ship Pred: ship	True: ship Pred: ship	True: airplane Pred: airplane
			
<pre>torch.save(model.state_dict(), "cifar10_resnet18_model.pth")</pre>			
<pre>print("Model saved successfully!")</pre>			
Model saved successfully!			
<pre>from google.colab import files</pre>			
<pre>files.download("cifar10_resnet18_model.pth")</pre>			

Conclusion

This project successfully implemented an image classification system using Deep Learning and Transfer Learning. The CIFAR-10 dataset was used, and images were preprocessed and augmented using resizing, horizontal flipping, rotation, and normalization. A pretrained ResNet18 model was used, with its final layer modified to classify the 10 CIFAR-10 classes.

The model was trained using PyTorch and evaluated using test accuracy, classification report, and confusion matrix. Training accuracy and loss were also visualized using graphs. The trained model was saved as a .pth file and used to make predictions on test images.

Overall, this task demonstrates how transfer learning can be applied to image classification and provides practical experience with data preprocessing, model training, evaluation, visualization, and model saving.